Reinforcement Learning

Chapter 4: Function Approximation

Ali Bereyhi

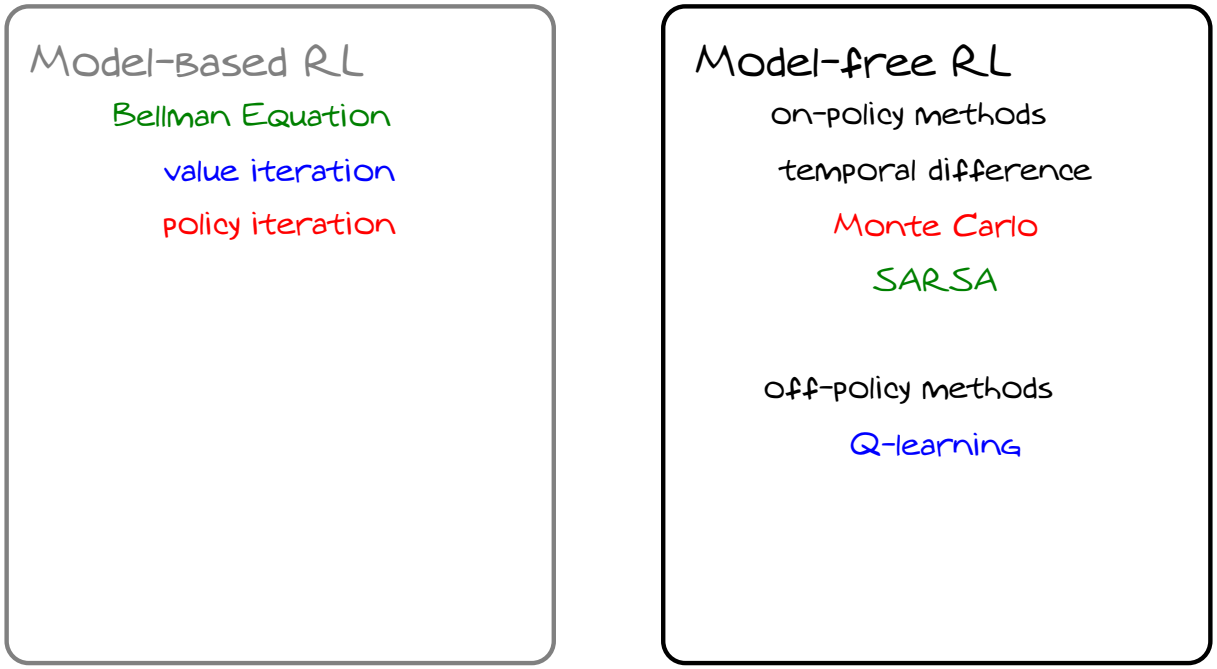
ali.bereyhi@utoronto.ca

Department of Electrical and Computer Engineering

University of Toronto

Fall 2026

Review: *Where Are We Now?*



What we have studied up to now is usually called tabular RL

- \+ *Why we call it tabular?*
- – *Because we think of value and action-value function as a table*

Tabular RL: *Schematic*

s^1	$\hat{v}_\pi\left(s^1\right)$
s^2	$\hat{v}_\pi\left(s^2\right)$
$\vdots$	$\vdots$
s^N	$\hat{v}_\pi\left(s^N\right)$

value-table

	a^1	a^2	$\ldots$	a^M
s^1	$\hat{q}_\pi\left(s^1, a^1\right)$	$\hat{q}_\pi\left(s^1, a^2\right)$		
s^2				
$\vdots$				
s^N				$\hat{q}_\pi\left(s^N, a^M\right)$

Q-table

Computational Complexity of Tabular RL

A tabular approach needs estimation of NM values from samples

- *Say we use Monte-Carlo*
  - ↳ *We need to visit all the state-action pairs enough times*
  - ↳ *Say enough is C for us; then, we need*

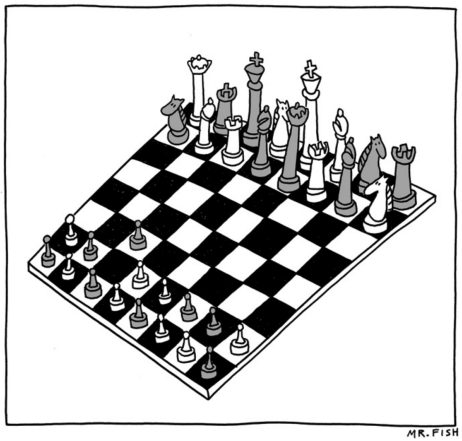
$$\#\text{ sample pairs in all episodes} \approx CNM$$

- *Same thing with temporal difference*

Moral of Story

In tabular RL methods, the number of required sample interactions with the environment scales with number of states and number of actions

Complexity Examples: *Chess*



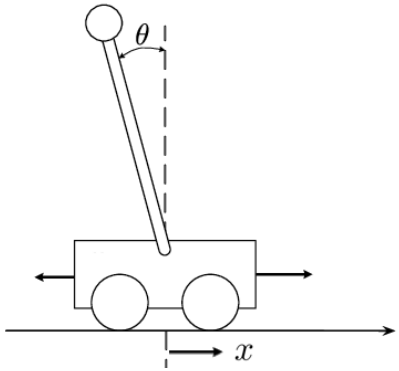
Shannon found out about roughly 10^{120} *possible states for chess*

- *Later on people came with other approximations*
  - *Let's take Tromp's number, i.e.,* $\approx 4 \times 10^{44}$
- *Say again we need only* 10^{-12} *sec per step and* $C = 10$

$$\text{time} \approx 4 \times 10^{33} M \text{ sec} \approx 12 \times 10^{25} M \text{ years}$$

- *Milky Way is about 13.6 billion years old!*

Complexity Example: *Continuous State Space*



We saw the Cart-Pole problem: *if we want to put it into a tabular form*

- ❓ *We need to put it into a table*
  - ✔ *We have to make a discrete grid of states*
  - ❓ *Say we have L state parameters and we grid each one with B bins*
    - 💡 *We have B^L grid points in total*
- ❗ *This grows exponentially large in number of state parameters!*

Computational Complexity of Tabular RL

In most interesting problems, we are dealing with scaling problem

- *We have an exponentially large number of states*
- *We cannot even visit a subset of them!*
- *Our tabular RL algorithms will not give us any meaningful results!*

Solution: *Function Approximation*

The solution is to use the value estimates of a few states to train a parameterized mode, e.g., a NN, that approximate the true value funciton

Approximating Value Function

We are given with a policy π and want to find some estimate $\hat{v}_\pi(\cdot)$

In the tabular RL, we assume that

$$v_\pi\left(s^1\right) = v^1 \qquad \ldots \qquad v_\pi\left(s^N\right) = v^N$$

for some unknown $v^1, \ldots, v^N$ and try to estimate them

Let us now approach the problem differently: *we assume that*

$$v_\pi(s) = f(s, \mathbf{w})$$

for some weights in $\mathbf{w}$ and a known function f

- $f()$ *is a approximation model that we assume for the value function*
- $\mathbf{w}$ *contains a set of learnable parameters*

Approximating Value Function

If we assume an approximation model: *we use our observations to find weight vector* $\mathbf{w}^\star$ *that fits this approximator best to our observations. We then set*

$$\hat{v}_\pi(s) = f(s, \mathbf{w}^\star)$$

- ? *How is it better than tabular then?!*
- Well! For lots of reasons

1. *We can use every single sample to update the estimate for all states*
   - ↳ *We use samples to fit* $\mathbf{w}$*: this impacts on the whole value function*
2. *We get some updating estimates for states that have not been visited*
   - ↳ *If we get the right* $\mathbf{w}$*: we get the estimator for all states*
3. *We can capture the impact of one state on others*

Approximating Value Function

- ? *But how can we find such approximators?!*
1. *Linear function approximators*
2. *Deep neural networks (DNNs)*
3. *Eigen-transforms, e.g., Fourier or Wavelets*
4. . . .

In this course, we focus only on parametric approximators

- *Linear function approximators*
- *DNNs*

Recap: Parameterized Function Approximation

1. *A set of sample input-outputs of a function that we do not know*

$$\mathbb{D} = \{(\mathbf{x}_i, \mathbf{y}_i) : i = 1, \ldots, N\}$$

   - ↳ $\mathbf{x}_i$ *is the input and* $\mathbf{y}_i$ *is the output*
   - ↳ *Let's denote the unknown function with* $g(\cdot)$, *i.e.,* $\mathbf{y}_i = g(\mathbf{x}_i)$

2. *We assume an approximating model for this unknown function*

$$\hat{\mathbf{y}} = f(\mathbf{x}|\mathbf{w})$$

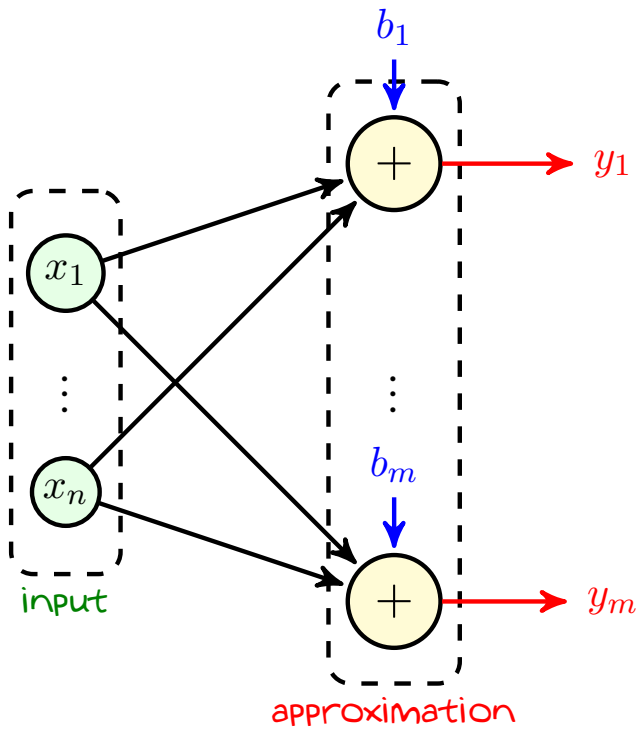
   - ↳ *In this approximator* $\mathbf{w}$ *is learnable, i.e., we are free to tune it as we wish*
   - ↳ *We treat output of this approximator as estimate of function outputs*

Example: *Linear Function Approximation*

A simple example of a function approximator is *the linear approximator*

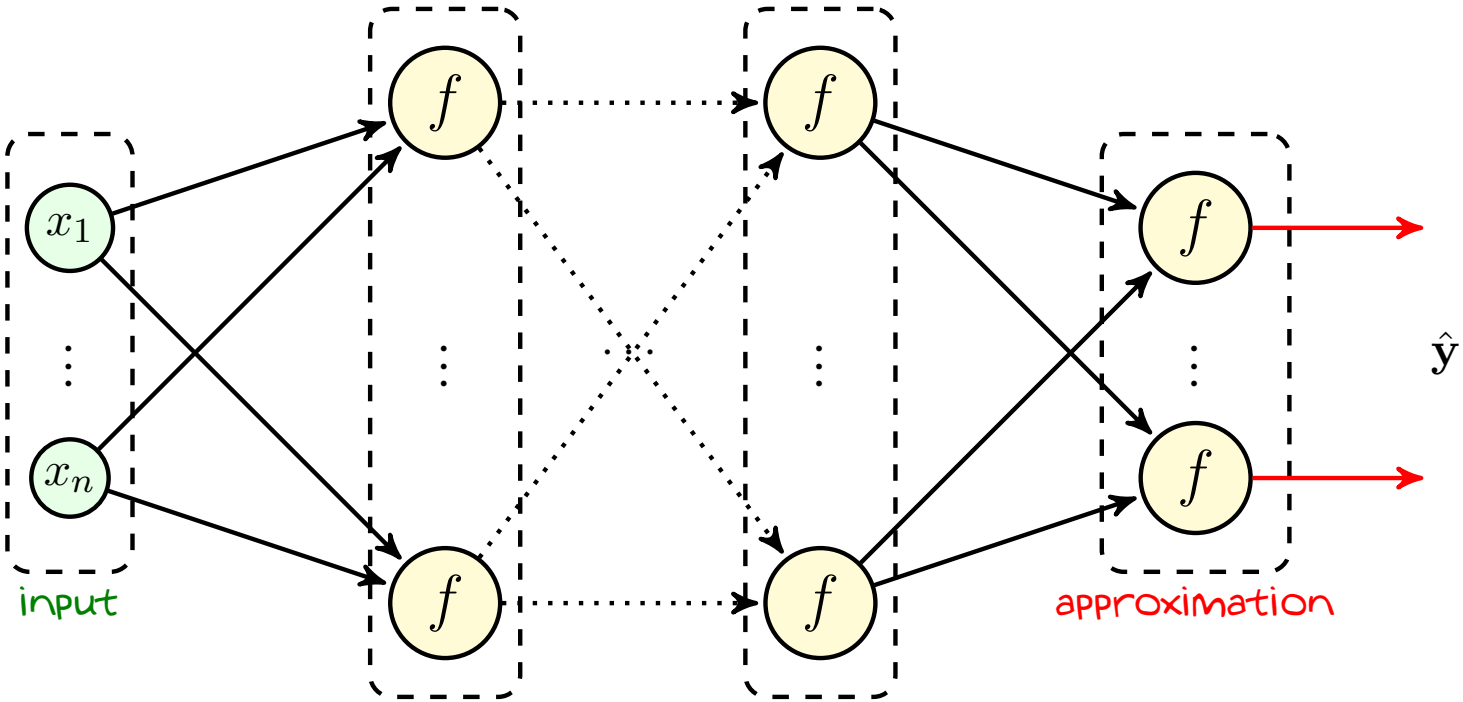
$$\hat{\mathbf{y}} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

where $\mathbf{W}$ *is the matrix of weights and* $\mathbf{b}$ *is the vector of biases*



Example: *Deep Neural Networks*

A deep neural network is also a parameterized approximator

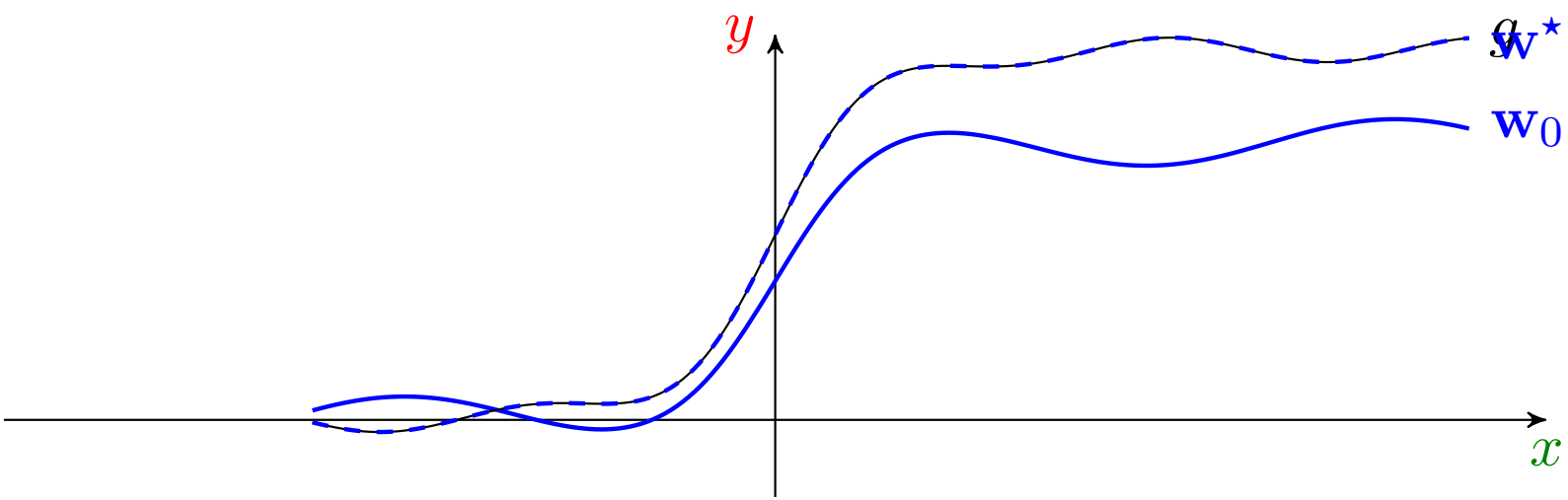


Example: *Deep Neural Networks*

- ? *Why should we rely on these approximators?*
- *They are known to be very expressive*

Universal Approximation Theorem

Given a family for neural networks: for any function g from its function space, there exists a sequence of configurations that approximates g arbitrarily precise



Gradient Descent

We use gradient descent for training: *we want to minimize*

$$\hat{R}(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\mathcal{L}\left(f\left(\mathbf{x}_i|\mathbf{w}\right),\mathbf{y}_i\right) \qquad \text{average loss}$$

`GradientDescent()`:

1: Initiate with some initial $\mathbf{w}^{(0)}$ and set a learning rate η
2: **while** not converged **do**
3: **for** $i = 1, \ldots, N$ **do**
4: Compute gradient for $\nabla_i = \nabla\mathcal{L}\left(f\left(\mathbf{x}_i|\mathbf{w}^{(t-1)}\right),\mathbf{y}_i\right)$
5: **end for**
6: Update weights as $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \eta\texttt{mean}\left\{\nabla_i\right\}$
7: **end while**

Stochastic Gradient Descent

To handle the complexity, we typically use stochastic GD

SGD():

1: Initiate with some initial $\mathbf{w}^{(0)}$ and set a learning rate η
2: **while** not converged **do**
3: **for** a random batch of size B **do**
4: Compute gradient for $\nabla_b = \nabla \mathcal{L}\left(f\left(\mathbf{x}_b | \mathbf{w}^{(t-1)}\right), \mathbf{y}_b\right)$
5: **end for**
6: Update weights as $\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t-1)} - \eta \texttt{mean}\left\{\nabla_i\right\}$ ← unbiased estimator
7: **end while**